BLOQUE II - TEMA 5

TECNOLOGÍA BÁSICA

Sistemas de gestión de bases de datos relacionales, orientados a objetos y NoSQL: características y componentes.

Supera. by thePower

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2024.

ÍNDICE

1. INTRODUCCIÓN A LOS SISTEMAS DE GESTIÓN DE BASES DE DATOS 5

1.1. Concepto de SGBD 5

1.2. Evolución histórica 5

1.3. Ventajas del uso de SGBD 5

2. BASES DE DATOS RELACIONALES 6

2.1. Modelo relacional: conceptos fundamentales 6

2.2. Reglas de Codd para SGBD Relacionales 8

2.3. Restricciones 9

2.3.1. Regla de integridad de entidad 9

2.3.2. Regla de integridad referencial 10

2.3.3. Regla de integridad de dominio 10

2.3.4. Restricciones de la base de datos 11

2.4. Lenguaje SQL 12

2.5. Normalización 12

2.5.1. Conceptos clave 12

2.5.2. Formas normales 13

2.6. Modelo ACID en bases de datos 14

2.7. SGBD relacionales 15

3. BASES DE DATOS ORIENTADAS A OBJETOS SGBDOO 15

3.1. Características de los SGBDOO 16

3.2. Conceptos del modelo orientado a objetos aplicados a BBDD 16

3.3. ORM (Mapeadores Objeto-Relacional) 17

3.4. Lenguajes de definición y consulta orientados a objetos 17

3.5. Identificadores y elementos usados en BDOO 18

3.6. Ejemplos de SGBD orientados a objetos 19

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

4. BASES DE DATOS NOSQL.....19

4.1. Origen y necesidades que cubren 19

4.2. Tipos de bases de datos NoSQL. Clasificación de las bases de datos NoSQL 20

4.3. Características comunes 22

4.4. Comparativa con bases de datos relacionales 23

4.5. Ejemplos de bases de datos NoSQL 23

Significado de los iconos que aparecen dentro de los TEMAS:



Examen


Importante


Recordatorio


Atención

Supera. by thePower

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

1. INTRODUCCIÓN A LOS SISTEMAS DE GESTIÓN DE BASES DE DATOS

1.1. CONCEPTO DE SGBD

Un **Sistema de Gestión de Bases de Datos (SGBD)**, o en inglés **DBMS (Database Management System)**, es un conjunto de programas que permiten **almacenar, organizar, manipular y recuperar datos** de forma eficiente y estructurada. Los SGBD actúan como intermediarios entre los usuarios y las bases de datos, permitiendo el acceso a los datos mediante lenguajes específicos como **SQL**.

1.2. EVOLUCIÓN HISTÓRICA

La evolución de los SGBD ha estado marcada por la necesidad de gestionar grandes volúmenes de información de forma más eficiente y flexible:

Etapa	Características
Década de 1960-70	Aparecen los primeros modelos de datos jerárquico (IMS) y en red (CODASYL).
Década de 1970-80	Se introduce el modelo relacional propuesto por E.F. Codd (1970) y aparecen los primeros SGBD relacionales, como Oracle (1979).
Década de 1980-90	Desarrollo de interfaces gráficas, transacciones, aparición de IBM DB2 (1983) y consolidación de SQL como estándar.
Década de 1990-2000	Aparecen los SGBD orientados a objetos y mejora la integración con lenguajes como Java.
Años 2000 en adelante	Nacen las bases de datos NoSQL para dar respuesta a nuevas necesidades de escalabilidad y flexibilidad en la web.
Actualidad	Uso de SGBD en la nube, bases de datos multimodelo, tecnologías Big Data y auge de las bases de datos vectoriales (pgvector, Pinecone, Weaviate, Milvus, Chroma) para aplicaciones de inteligencia artificial generativa y búsqueda semántica (RAG).

1.3. VENTAJAS DEL USO DE SGBD

El uso de sistemas de gestión de bases de datos aporta múltiples beneficios frente a métodos tradicionales como los archivos planos:

1. **Independencia lógica y física de los datos**: Los programas no dependen directamente de la estructura física de los datos.
2. **Reducción de redundancia e inconsistencia**: Al centralizar los datos, se evita duplicación y errores.
3. **Integridad y validación de datos**: Se pueden definir reglas y restricciones para garantizar que los datos son correctos.
4. **Acceso concurrente y controlado**: Permiten que varios usuarios trabajen a la vez sin conflictos.
5. **Seguridad**: Control de permisos por usuario, autenticación y cifrado.
6. **Recuperación ante fallos**: Mecanismos de backup y restore automáticos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

7. **Eficiencia en las consultas**: Optimizadores de consulta y uso de índices para mejorar el rendimiento.
8. **Escalabilidad y mantenimiento centralizado**: Adaptación a sistemas distribuidos, grandes volúmenes y entornos en la nube.

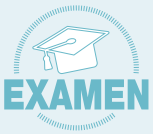
2. BASES DE DATOS RELACIONALES

El modelo relacional, además de estructurar los datos en forma de tablas, se basa en un conjunto de conceptos y restricciones que garantizan la consistencia, integridad y validez de los datos almacenados en la base de datos. Estos elementos son esenciales para que el sistema pueda operar de manera coherente ante múltiples usuarios y operaciones simultáneas.

2.1. MODELO RELACIONAL: CONCEPTOS FUNDAMENTALES

El **modelo relacional** fue propuesto por **E. F. Codd en 1970** y se basa en una estructura matemática simple: la **relación**, que se representa mediante **tablas**. Los conceptos clave del modelo relacional son:

- **Relación**: Es una **tabla** que representa una entidad del mundo real (por ejemplo, una tabla Clientes o Pedidos). Cada relación:
  - Tiene un **nombre único**.
  - Está formada por un conjunto de **atributos** (columnas).
  - Contiene **tuplas** (filas), cada una de las cuales representa una instancia concreta de la entidad.
- **Tupla**: Corresponde a **una fila de la tabla**, es decir, a un **registro**. Cada tupla representa una instancia concreta de la entidad (por ejemplo, un cliente con sus datos).
- **Atributo**: Cada **columna de la tabla** representa un atributo o campo de la entidad (como nombre, edad, correo).
  - Pertenecen al mismo tipo de dato (número, texto, fecha...).
  - Comparten el mismo **dominio**.



- **Clave primaria (primary key)**: Es un atributo o combinación de atributos que **identifica de manera única** a cada tupla en la tabla. Debe ser **no nulo** y **único**. Por ejemplo, el campo DNI en una tabla de personas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Clave foránea (foreign key)**: Es un campo que **enlaza una tabla con otra**, apuntando a la clave primaria de otra relación y **garantiza la integridad referencial,** es decir, establece relaciones entre entidades. Por ejemplo, id_cliente en la tabla Pedidos es una clave foránea que apunta a Clientes.
- **Dominio**: Es el **conjunto de valores válidos** que puede tomar un atributo. Por ejemplo, un dominio para el campo edad puede ser los números enteros entre 0 y 120.
- **Grado**: El **grado** de una relación es el **número de atributos** (columnas) que contiene. Por ejemplo, si la tabla Clientes tiene los atributos id, nombre, email, su grado es **3**.
- **Cardinalidad:** La **cardinalidad** de una relación es el **número de tuplas** (filas) que contiene. Por ejemplo, si Clientes tiene 250 registros, su cardinalidad es **250**. No debe confundirse con la cardinalidad de una relación en el modelo entidad-relación (1:1, 1:N, N:M), que describe cuántas instancias de una entidad se asocian con otra; ambos conceptos comparten nombre pero tienen significados distintos.

El modelo relacional se apoya en principios de **normalización**, que consisten en organizar los datos para reducir la redundancia y asegurar la integridad.

Ejemplo: Supongamos una base de datos con dos relaciones: `Clientes y Pedidos.` Para este caso tendríamos que:

- id_cliente es **clave primaria** en Clientes.
- id_cliente es **clave foránea** en Pedidos, que vincula cada pedido con un cliente.

Clientes

id_cliente	nombre	correo
1	Ana Torres	ana@email.com
2	Luis García	luis@email.com

Pedidos

id_pedido	fecha	id_cliente
101	2024-03-10	1
102	2024-03-15	2

El modelo relacional se apoya en las reglas de **normalización**, un conjunto de principios cuyo objetivo es:

- **Reducir la redundancia** (eliminación de datos duplicados).
- **Asegurar la integridad** de los datos mediante dependencias claras.
- **Mejorar la eficiencia y mantenibilidad** de la base de datos.

Las formas normales (1FN, 2FN, 3FN, etc.) definen niveles de organización cada vez más estrictos y ordenados para garantizar la calidad del modelo de datos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

RECUERDA:
El Modelo RELACIONAL, se basa en relaciones (tablas) formadas por:

- Atributos (columnas): características de la entidad.
- Tuplas (filas): registros concretos.
- Dominio: conjunto de valores válidos por atributo.
- Clave primaria: identifica de forma única cada tupla.
- Clave foránea: enlaza tablas, garantizando la integridad referencial.
- Grado: nº de atributos; Cardinalidad: nº de tuplas.

2.2. REGLAS DE CODD PARA SGBD RELACIONALES



Las reglas de Codd, formuladas por Edgar F. Codd en 1985, establecen un conjunto de 13 principios que deben cumplir los Sistemas de Gestión de Bases de Datos para ser considerados plenamente relacionales. Estas reglas sirven como referencia teórica para evaluar el grado de conformidad de un SGBD con el modelo relacional.

Regla		Descripción
Regla 0	**Sistema relacional**	Todo sistema debe usar únicamente capacidades relacionales para gestionar datos.
Regla 1	**Representación de la información**	Toda información debe almacenarse como valores en tablas.
Regla 2	**Acceso garantizado**	Cada dato debe poder ser accedido mediante el nombre de tabla, clave y atributo.
Regla 3	**Tratamiento de valores nulos**	Deben gestionarse de forma lógica los valores desconocidos o no aplicables (NULL).
Regla 4	**Diccionario de datos relacional**	El catálogo (metadatos) debe poder consultarse usando el mismo modelo relacional.
Regla 5	**Lenguaje completo de datos**	Debe existir un lenguaje que permita definición, manipulación y control de datos (ej. SQL).
Regla 6	**Actualización mediante vistas**	Las vistas deben poder actualizarse si su definición lo permite lógicamente.
Regla 7	**Manipulación en conjunto**	Las operaciones deben poder aplicarse a conjuntos de registros, no solo individuales.
Regla 8	**Independencia física**	Cambios físicos en almacenamiento no deben afectar a las aplicaciones.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Regla		Descripción
Regla 9	**Independencia lógica**	Cambios en la estructura lógica no deben requerir cambios en los programas.
Regla 10	**Integridad declarativa**	Las reglas de integridad deben estar definidas en la base de datos, no en el código.
Regla 11	**Independencia de distribución**	El comportamiento debe ser igual con datos centralizados o distribuidos.
Regla 12	**No subversión**	No debe poder eludirse el modelo relacional desde niveles bajos del sistema.

2.3. RESTRICCIONES

Las **restricciones** son reglas que el sistema de gestión de bases de datos (SGBD) **impone sobre los datos** para garantizar su **validez, coherencia e integridad** a lo largo del tiempo. Estas reglas aseguran que los datos almacenados cumplan ciertas condiciones lógicas y evitan errores, duplicidades o inconsistencias.

Tipos de restricciones comunes:

1. **Restricción de dominio**
   Cada atributo debe tomar un valor que pertenezca a su **dominio definido** (por ejemplo, un campo "edad" solo puede aceptar números enteros positivos).
2. **Restricción de unicidad**
   Un atributo (o conjunto de atributos) no puede contener valores duplicados. Es típica en claves primarias y en claves candidatas.
3. **Restricción de nulidad (NOT NULL)**
   Determina si un campo puede aceptar valores nulos. Por ejemplo, una clave primaria **no puede ser nula**.
4. **Clave primaria (PRIMARY KEY)**
   Garantiza que cada tupla de la tabla esté **únicamente identificada** por uno o varios atributos. Implica **unicidad** y **no nulidad**.
5. **Clave foránea (FOREIGN KEY)**
   Es un campo o conjunto de campos que **referencia** a la clave primaria de otra tabla, asegurando la **integridad referencial** entre entidades relacionadas.
6. **Restricción de integridad referencial**
   Evita que existan registros huérfanos. Por ejemplo, no se puede insertar un pedido con un id_cliente que no exista en la tabla Clientes.
7. **Restricciones definidas por el usuario (CHECK)**
   Permiten establecer condiciones personalizadas, como CHECK (edad >= 18).

2.3.1. Regla de integridad de entidad

Todo valor de una **clave primaria** debe ser:

- **Único**: no puede repetirse.
- **No nulo**: debe estar siempre presente.

—> Esto garantiza que cada fila pueda ser identificada de forma unívoca.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Ejemplo:

```
CREATE TABLE Clientes (
  id_cliente INT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL
);
```

2.3.2. Regla de integridad referencial

Establece que los valores de una **clave foránea** deben:

- Coincidir con valores existentes en la clave primaria de otra tabla.
- O ser nulos (si está permitido).

–> Esto asegura la **coherencia entre tablas relacionadas**.

Ejemplo:

```
CREATE TABLE Pedidos (
  id_pedido INT PRIMARY KEY,
  id_cliente INT,
  FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)
);
```

2.3.3. Regla de integridad de dominio

La **regla de integridad de dominio** establece que **cada valor almacenado en un atributo debe pertenecer a un conjunto válido de valores predefinidos**, es decir, a su **dominio**. Esta restricción garantiza que los datos introducidos sean **coherentes con el tipo y formato esperado**, y evita errores de entrada y ambigüedades.

Por ejemplo, si un atributo correo debe contener direcciones válidas, no se permitirán números o cadenas que no cumplan con el formato de un correo electrónico.



RECUERDA:

Un dominio es el conjunto de valores válidos que puede tomar un atributo. Puede estar definido por:

- Un tipo de dato (por ejemplo, INT, VARCHAR, DATE, BOOLEAN),
- Una restricción lógica (por ejemplo, edad >= 0),
- O un conjunto específico de valores (por ejemplo, estado IN ('Activo', 'Inactivo')).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Los SGBD relacionales permiten aplicar esta regla mediante varios mecanismos:

- **Tipos de datos**: restringen automáticamente el tipo de valores

  Ejemplo: VARCHAR(50) solo admite texto de hasta 50 caracteres
- **Restricciones CHECK**: definen condiciones lógicas personalizadas.

  Ejemplo: CHECK (edad >= 18)
- **Restricciones de lista (ENUM)**: en algunos SGBD, permiten definir un conjunto fijo de valores.
  Ejemplo en MySQL: estado ENUM('Activo', 'Inactivo', 'Pendiente')
- **Expresiones regulares** (REGEXP)**: útiles para validar formatos complejos, como correos electrónicos o números de teléfono.
- **Procedimientos o triggers**: validaciones más complejas cuando se necesita lógica adicional.

2.3.4. Restricciones de la base de datos

Además de las reglas anteriores, se pueden definir **otras restricciones a nivel de base de datos**, como:

- **Restricciones de unicidad** (UNIQUE): aseguran que no haya valores duplicados en un campo que no es clave primaria.
- **Valores por defecto** (DEFAULT): asignan un valor inicial si no se especifica otro.
- **Valores obligatorios** (NOT NULL): evitan que un campo quede sin valor.
- **Restricciones de validación compuesta**: expresiones complejas que afectan a múltiples campos.

Restricción	Descripción	Ejemplo
UNIQUE	Asegura que los valores de un atributo (o combinación de ellos) sean únicos, aunque no sea clave primaria.	email UNIQUE en una tabla de usuarios.
DEFAULT	Define un valor por defecto que se aplicará si no se proporciona uno al insertar una tupla.	estado DEFAULT 'activo'
NOT NULL	Obliga a que el atributo no pueda tener valores nulos. Es común en campos obligatorios.	nombre NOT NULL
CHECK (validación compuesta)	Permite definir condiciones lógicas complejas que deben cumplirse al insertar o actualizar datos.	CHECK (edad >= 18 AND pais = 'España')
Restricciones múltiples	Se pueden combinar restricciones para mayor precisión.	telefono VARCHAR(9) UNIQUE NOT NULL CHECK (telefono LIKE '6%')

¿Dónde se aplican estas restricciones?

- A nivel de **columna** (por ejemplo, NOT NULL, DEFAULT, CHECK).
- A nivel de **tabla** (por ejemplo, UNIQUE en combinación de columnas, restricciones entre campos).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.4. LENGUAJE SQL

El **SQL (Structured Query Language)** es el **lenguaje estándar de comunicación con bases de datos relacionales**. Fue desarrollado por IBM en los años 70 y adoptado posteriormente como estándar por organismos como ANSI e ISO. Su sintaxis es declarativa, lo que significa que el usuario indica **qué desea obtener**, y no cómo debe hacerse.

SQL permite definir, consultar, modificar y administrar los datos de forma uniforme, siendo común a la mayoría de los SGBD relacionales, como **Oracle, MySQL, PostgreSQL, SQL Server**, entre otros.

SQL se organiza en varios **subconjuntos** según el tipo de operaciones que permite realizar:



Tipo	Propósito	Ejemplos
DDL (Data Definition Language)	Definición de estructuras	**CREATE TABLE** **ALTER** **DROP**
DML (Data Manipulation Language)	Manipulación de datos	**INSERT** **UPDATE** **DELETE**
DQL (Data Query Language)	Consulta de datos	**SELECT**
DCL (Data Control Language)	Control de acceso	**GRANT** **REVOKE**
TCL (Transaction Control Language)	Control de transacciones	**COMMIT** **ROLLBACK** **SAVEPOINT**

2.5. NORMALIZACIÓN

La **normalización** es un proceso teórico utilizado en el diseño de bases de datos relacionales para **organizar los datos de manera eficiente**, **reducir la redundancia** y **garantizar la integridad**.

Se basa en aplicar una serie de reglas denominadas **formas normales**, que permiten dividir una base de datos compleja en **relaciones más simples**, sin pérdida de información.

2.5.1. Conceptos clave

En el proceso de normalización, las **dependencias funcionales** juegan un papel fundamental para identificar cómo se relacionan los atributos dentro de una tabla. Comprender sus tipos permite diseñar bases de datos coherentes y sin redundancias.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Dependencia funcional** es una relación lógica entre atributos de una tabla que indica que **un valor de un atributo (o conjunto de atributos) determina unívocamente el valor de otro**.
    - Se denota como:
      X →Y
    - Se lee: **"Y depende funcionalmente de X" o "X determina Y".**
    - Esto significa que **si dos tuplas tienen el mismo valor en X, entonces deben tener el mismo valor en Y**.
- **Dependencia Funcional Reflexiva.** Si Y está contenido en X, entonces X → Y. **Esto se basa en la lógica: un conjunto determina cualquier subconjunto de sí mismo.**
- **Dependencia Funcional Aumentativa (o de Unión/Augmentación).** Si X → Y, entonces XZ → YZ
- **Dependencia Funcional Transitiva.** Si X → Y y Y → Z, entonces X → Z

2.5.2. Formas normales

Las **formas normales** son etapas progresivas del proceso de normalización que permiten estructurar una base de datos de forma eficiente. Cada forma normal impone reglas más estrictas que la anterior para eliminar redundancias, evitar anomalías y asegurar la integridad de los datos.

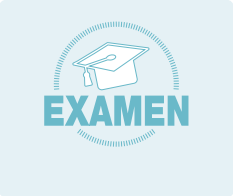
Forma Normal	Condición	Objetivo principal
1NF	Todos los atributos deben ser **atómicos** (no repetidos ni multivaluados).	Eliminar grupos repetitivos y garantizar estructura tabular.
2NF	Estar en 1NF + **eliminar dependencias parciales** (atributos que dependen solo de parte de la clave primaria compuesta).	Separar los datos que no dependen totalmente de la clave.
3NF	Estar en 2NF + eliminar **dependencias transitivas** (un atributo depende de otro que no es clave).	Aislar dependencias no directas respecto a la clave primaria.
BCNF	Variante más estricta de 3NF: para toda dependencia funcional A → B, A debe ser una **superclave**.	Asegura una descomposición totalmente libre de anomalías.
4NF	Eliminar **dependencias multivaluadas.**	Separar valores múltiples en relaciones independientes.
5NF	Descompone relaciones complejas con **dependencias de unión** (join dependency).	Reconstrucción sin pérdida desde subconjuntos de datos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.6. MODELO ACID EN BASES DE DATOS

El modelo **ACID** es un conjunto de propiedades que debe cumplir cualquier sistema de gestión de bases de datos transaccional para **garantizar la integridad, coherencia y fiabilidad** de los datos, incluso ante fallos o accesos simultáneos.

El acrónimo **ACID** representa las siguientes cuatro propiedades fundamentales:



A	Atomicidad
C	Consistencia
I	Isolation (Aislamiento)
D	Durabilidad

A – Atomicidad
Una **transacción** es una unidad indivisible de trabajo: o se ejecuta **completamente** o **no se ejecuta en absoluto**.

- Si ocurre un fallo durante la ejecución, el sistema **revierte** todos los cambios.
- Garantiza que no queden datos en un estado intermedio o inconsistente.

C – Consistencia

Una transacción debe llevar la base de datos de un estado **válido** a otro **estado válido**, respetando todas las reglas de integridad definidas (como claves primarias, foráneas, restricciones...).

- Asegura que los datos **nunca violen las reglas del esquema**.
- Se mantienen relaciones válidas entre entidades.

I – Aislamiento (*Isolation*)

Las transacciones concurrentes deben ejecutarse como si **fueran secuenciales**, sin interferencias entre sí.

- Los cambios de una transacción no son visibles para otras hasta que finaliza.
- Previene problemas como lecturas sucias, no repetibles o fantasmas.

D – Durabilidad

Una vez que una transacción se **confirma (commit)**, sus cambios **permanecen**, incluso si el sistema falla inmediatamente después.

- El sistema asegura la persistencia de los datos mediante logs, backups o escritura en disco.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2.7. SGBD RELACIONALES

Existen múltiples sistemas que implementan el modelo relacional. Algunos de los más utilizados son:



SGBD	Características destacadas
MySQL	Código abierto, ligero y ampliamente usado en desarrollo web (LAMP). MySQL es propiedad de Oracle Corporation desde su adquisición de Sun Microsystems en 2010; esto motivó la creación de MariaDB como fork comunitario independiente.
PostgreSQL	Es un sistema de código abierto (Open Source), avanzado, extensible y con fuerte cumplimiento de estándares. Ideal para soluciones críticas.
Oracle Database	Comercial, robusto, con soporte para grandes volúmenes y seguridad avanzada.
SQL Server	Solución de Microsoft, integrada con herramientas como .NET, Power BI y Azure.
MariaDB	Derivado de MySQL, con enfoque en comunidad y compatibilidad.
SQLite	Motor embebido, sin proceso de servidor independiente; es, según sus propios desarrolladores, la base de datos más ampliamente desplegada del mundo (apps móviles, navegadores, sistemas embebidos).

3. BASES DE DATOS ORIENTADAS A OBJETOS SGBDOO

Las **bases de datos orientadas a objetos (BBOO)** surgen como una evolución del modelo relacional para integrar las ventajas de la **programación orientada a objetos** con la persistencia de datos. Están diseñadas para almacenar objetos completos (con atributos y métodos), tal como se definen en lenguajes como Java, C++ o Python.

Una **característica fundamental** de los **sistemas de gestión de bases de datos orientadas a objetos** es su capacidad para representar y manipular datos complejos mediante la integración de conceptos de la programación orientada a objetos, como clases, herencia, encapsulamiento y polimorfismo. Estos sistemas permiten construir objetos complejos, asignarles un identificador único independiente de sus atributos y estructurar el esquema en clases o tipos.

Este tipo de bases de datos es especialmente útil en entornos donde los datos tienen una **estructura compleja, jerárquica o interrelacionada**, como en ingeniería, CAD, multimedia, GIS o simulación.

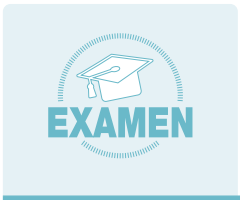


Sistema de gestión de base de datos que representa la información en forma de objetos tal y como son utilizados en la programación orientada a objetos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3.1. CARACTERÍSTICAS DE LOS SGBDOO

El **modelo orientado a objetos** es un paradigma de programación y diseño que organiza el software en torno a "objetos", que combinan datos (atributos) y comportamiento (métodos). Sus características fundamentales son:



Característica	Descripción
Modularidad	Permite dividir el sistema en partes independientes (objetos o clases).
Abstracción	Oculta los detalles internos y muestra solo lo esencial para el usuario.
Reusabilidad	Permite reutilizar clases y objetos mediante mecanismos como la herencia.
Encapsulamiento	Los datos están protegidos dentro del objeto y solo accesibles mediante métodos definidos.

3.2. CONCEPTOS DEL MODELO ORIENTADO A OBJETOS APLICADOS A BBDD

Las **bases de datos orientadas a objetos (OODB)** aplican los principios de la **programación orientada a objetos (POO)** al almacenamiento y gestión de datos persistentes. En lugar de trabajar con tablas y filas, estas bases de datos almacenan **objetos complejos** que pueden tener atributos y métodos, y mantener relaciones jerárquicas y de herencia.

Conceptos clave:

- **Objeto**: entidad que encapsula datos (atributos) y comportamientos (métodos).
- **Clase**: plantilla a partir de la cual se crean objetos.
- **Herencia**: capacidad de una clase para derivar de otra y heredar sus propiedades.
- **Encapsulamiento**: los datos de un objeto solo pueden modificarse a través de sus métodos.
- **Polimorfismo**: objetos de diferentes clases pueden responder al mismo mensaje de forma distinta.

Ventajas e inconvenientes frente a modelos relacionales

Ventajas	Inconvenientes
Se elimina la necesidad de realizar **mapeo objeto-relacional (ORM).**	Menor adopción y estandarización que los SGBD relacionales.
Mayor naturalidad al trabajar con lenguajes orientados a objetos.	Rendimiento más bajo en operaciones simples comparado con SQL.
Ideal para representar datos **jerárquicos o complejos** (multimedia, CAD).	Escasa compatibilidad con herramientas y entornos tradicionales.
Soporte a encapsulamiento, herencia y polimorfismo en el modelo de datos.	Lenguajes de consulta y definición menos estandarizados.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3.3. ORM (MAPEADORES OBJETO-RELACIONAL)

Un **ORM** (*Object-Relational Mapping*) es una técnica que permite mapear clases de programación orientada a objetos con tablas de bases de datos relacionales, facilitando la persistencia de objetos sin escribir consultas SQL manualmente. Los ORMs permiten trabajar con bases de datos utilizando el mismo lenguaje de programación del sistema (Java, C#, Python, etc.).

Características de los ORM:

- Automatizan la conversión entre objetos y registros de tablas.
- Permiten realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sin escribir SQL directamente.
- Facilitan la portabilidad del código entre distintos motores de bases de datos.
- Mejoran la mantenibilidad y productividad del desarrollo.

Ejemplos de ORM:

- **Hibernate** (Java)
- **NHibernate** (C#/.NET, inspirado en Hibernate)
- **Entity Framework** (ADO.NET - Microsoft .NET)
- **Ruby on Rails ActiveRecord** (Ruby)
- **SQLAlchemy y Django ORM** (Python)
- **Prisma** (Node.js/TypeScript)

3.4. LENGUAJES DE DEFINICIÓN Y CONSULTA ORIENTADOS A OBJETOS

Los SGBDOO utilizan extensiones orientadas a objetos del SQL o lenguajes específicos:

- **OQL (Object Query Language)**: lenguaje estándar propuesto por el consorcio ODMG (Object Data Management Group), similar a SQL pero adaptado al paradigma objeto.
- **JPQL (Java Persistence Query Language)**: utilizado con frameworks como JPA o Hibernate.
- Extensiones orientadas a objetos en algunos SGBD relacionales (como Oracle o PostgreSQL con tipos definidos por el usuario).



Uno de los lenguajes más utilizados es el **OQL (Object Query Language)**, un lenguaje de consulta **inspirado en SQL pero adaptado al paradigma orientado a objetos**.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Permite realizar consultas sobre objetos, colecciones, atributos complejos y relaciones jerárquicas.
- Soporta operaciones como *select*, *where*, *group by*, *order by*, pero puede navegar a través de atributos de tipo objeto o colección.

3.5. IDENTIFICADORES Y ELEMENTOS USADOS EN BDOO

1. OID (Object Identifier) - Obligatorio

- **Definición**: Es un **identificador único** asignado automáticamente a cada objeto en una BDOO cuando se crea.
- **Propósito**: Permite **referenciar el objeto directamente**, sin necesidad de buscarlo por atributos o valores de campos (como se hace con claves primarias en bases relacionales).
- **Persistencia**: El OID se mantiene constante mientras el objeto exista en la base de datos.
- **Transparencia**: Los usuarios normalmente no interactúan directamente con el OID, pero el sistema lo usa internamente para navegación y referencias entre objetos.

2. OID derivado de atributos (opcional)

- En algunos sistemas, **se puede definir un identificador compuesto por uno o más atributos del objeto** (por ejemplo, un DNI o ID de producto).
- Es similar a una **clave primaria** en bases de datos relacionales.
- Aunque no sustituye al OID, **puede usarse para búsquedas o restricciones de unicidad**.

3. Referencias (object references)

- Atributos especiales que **apuntan directamente al OID de otro objeto**.
- Se usan para modelar relaciones entre objetos (como punteros o enlaces).
- Más eficientes que claves foráneas en relaciones complejas o jerárquicas.

4. Nombre o identificador simbólico (nombre lógico)

- Algunas BDOO permiten asignar un **nombre simbólico** (como un alias) a un objeto para acceder a él más fácilmente.
- Útil en entornos donde se requiere identificar objetos por nombre y no solo por OID (por ejemplo: objeto_usuario['juan']).

5. Identificadores de clase y subclase

- Cada objeto **pertenece a una clase** (o a una subclase si hay herencia).
- Esto permite identificar el **tipo** del objeto, aunque no sea único por sí mismo.
- Útil para consultas y restricciones por tipo.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3.6. EJEMPLOS DE SGBD ORIENTADOS A OBJETOS



SGBDOO	Descripción
ObjectDB	Base de datos para Java, compatible con JPA. Ligera y de alto rendimiento.
db4o	OODB para Java y .NET que almacenaba objetos directamente sin necesidad de mapeo. Proyecto discontinuado: Versant (propietaria desde 2008) dejó de darle soporte activo hacia 2014; no se recomienda para nuevos desarrollos.
Versant	Plataforma robusta y escalable, usada en aplicaciones industriales y científicas. Tras la adquisición de Versant Corporation por Actian en 2013, el producto se comercializa actualmente como Actian NoSQL Object Database.
Matisse	Sistema orientado a objetos con integración SQL y OQL.

4. BASES DE DATOS NOSQL

4.1. ORIGEN Y NECESIDADES QUE CUBREN

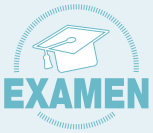
Las bases de datos **NoSQL** (Not Only SQL) surgieron a partir de las nuevas necesidades de **almacenamiento masivo de datos no estructurados**, escalabilidad horizontal y disponibilidad constante, especialmente en contextos como redes sociales, comercio electrónico, IoT y Big Data.

Su aparición se justifica por:

- Limitaciones de las bases de datos relacionales en entornos distribuidos y de alto rendimiento.
- Necesidad de **mayor flexibilidad en el esquema** de datos.
- Requerimientos de **alta escalabilidad y disponibilidad** en tiempo real.
- Almacenamiento de grandes volúmenes de datos heterogéneos, semiestructurados o sin estructura.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

4.2. TIPOS DE BASES DE DATOS NOSQL. CLASIFICACIÓN DE LAS BASES DE DATOS NOSQL



Las bases de datos NoSQL no utilizan el modelo relacional tradicional y se han diseñado para ofrecer mayor escalabilidad, flexibilidad y rendimiento ante grandes volúmenes de datos y estructuras poco definidas. **Se clasifican principalmente en función del modelo de datos** que emplean:

a) Bases de datos clave-valor

- Cada registro se almacena como un par **clave : valor**, donde la clave es única y el valor puede ser cualquier dato (texto, objeto, binario, etc.).
- **Ventajas**:
  - Extremadamente rápidas en operaciones de lectura y escritura.
  - Muy eficientes para cachés, sesiones y perfiles de usuario.
  - Fácil escalabilidad horizontal.
- **Limitaciones**: No están pensadas para relaciones complejas entre datos.
- **Ejemplos**:
  - **Redis** (en memoria, con soporte para estructuras de datos avanzadas),
  - **Riak** (enfocada a alta disponibilidad),
  - **Amazon DynamoDB** (gestión completamente administrada en la nube de AWS).

b) Bases de datos documentales

- Los datos se almacenan como documentos estructurados, usualmente en formato **JSON**, **BSON** o **XML**, permitiendo una gran flexibilidad.
- **Ventajas**:
  - Admiten estructuras complejas y anidadas.
  - Permiten consultas eficientes sobre los campos del documento.
  - Ideales para aplicaciones con datos semiestructurados.
- **Limitaciones**: Puede haber cierta duplicación de datos si no se diseña cuidadosamente.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Ejemplos**:
  - » **MongoDB** (el más popular y ampliamente usado),
  - » **CouchDB** (centrado en la replicación y sincronización de datos),
  - » **RavenDB** (enfocado en integraciones .NET).

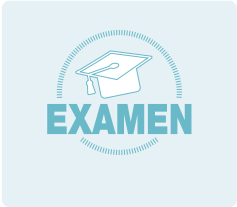
c) Bases de datos en columnas

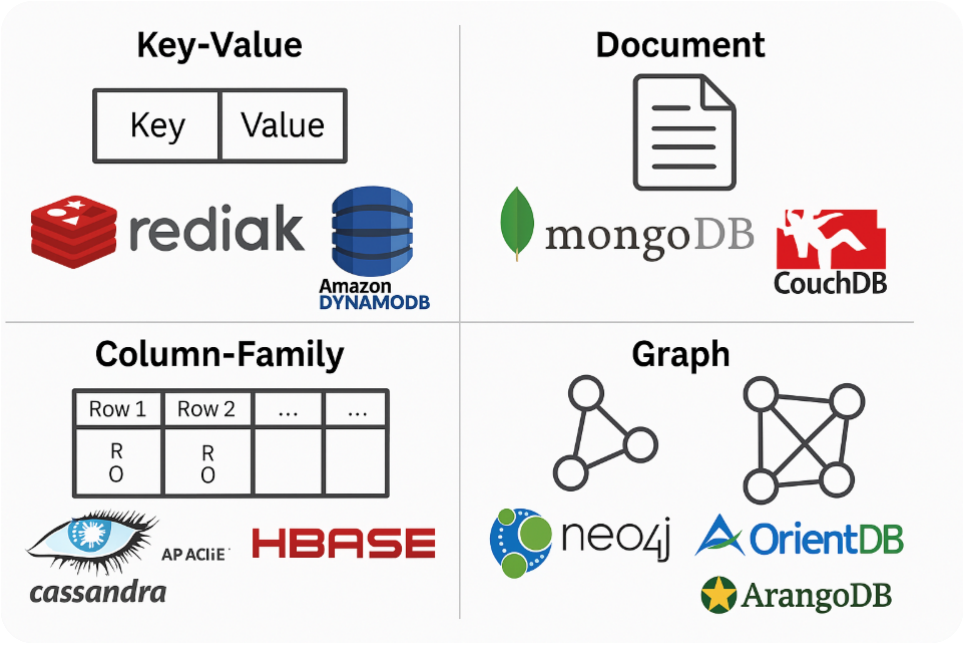
- Organizan los datos por columnas en lugar de filas, agrupando columnas relacionadas en "familias de columnas".
- **Ventajas**:
  - » Alto rendimiento para consultas analíticas y procesamiento masivo.
  - » Optimización para lectura y escritura de grandes volúmenes de datos.
  - » Ideal para **Big Data** y análisis por lotes.
- **Limitaciones**: Más complejas de modelar correctamente que las bases relacionales o documentales.
- **Ejemplos**:
  - » **Apache Cassandra** (altamente escalable y tolerante a fallos),
  - » **HBase** (construida sobre Hadoop, buena integración con MapReduce),
  - » **ScyllaDB** (compatibilidad con Cassandra pero con mejor rendimiento).

d) Bases de datos basadas en grafos

- Representan los datos como nodos (entidades) y aristas (relaciones), permitiendo expresar conexiones complejas.
- **Ventajas**:
  - » Muy eficientes para recorrer relaciones entre datos.
  - » Excelentes para aplicaciones con redes sociales, recomendaciones, rutas, jerarquías, etc.
- **Limitaciones**: Menos adecuadas para cargas transaccionales simples.
- **Ejemplos**:
  - » **Neo4j** (el referente en bases de datos de grafos),
  - » **OrientDB** (combinación entre grafos y documentos),
  - » **ArangoDB** (multi-modelo: grafos, documentos y clave-valor).

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025





4.3. CARACTERÍSTICAS COMUNES

Aunque los sistemas NoSQL varían en estructura y propósito (clave-valor, documental, grafos, columnas...), comparten una serie de características generales que los diferencian de los SGBD relacionales tradicionales:

Característica	Descripción
Escalabilidad horizontal	Permiten añadir nodos al sistema fácilmente para mejorar el rendimiento.
Alta disponibilidad	Tolerancia a fallos y replicación de datos para garantizar el acceso continuo.
Modelo BASE	Alternativa al modelo ACID
	- **Basically Available** (disponibilidad básica)
	- **Soft state** (el estado puede cambiar con el tiempo)
	- **Eventually consistent** (consistencia eventual, no inmediata)
Flexibilidad de esquema	Permiten estructuras dinámicas o sin esquema fijo.
Alto rendimiento	Muy optimizadas para lecturas y escrituras masivas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Modelo BASE (frente al modelo ACID)

A diferencia del modelo ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) propio de las bases de datos relacionales, muchas bases NoSQL siguen el modelo **BASE**, que responde mejor a entornos distribuidos, escalables y con grandes volúmenes de datos:

BASE	Significado
Basically Available	El sistema está siempre disponible, incluso ante fallos parciales.
Soft state	El estado del sistema puede cambiar con el tiempo, incluso sin nuevas entradas, debido a la replicación asincrónica.
Eventually consistent	Los datos se propagan entre nodos de manera que eventualmente todos los nodos tendrán la misma información.

BASE no garantiza una consistencia inmediata, sino que apuesta por la consistencia eventual, lo que permite una mejor escalabilidad y tolerancia a fallos en entornos distribuidos.

4.4. COMPARATIVA CON BASES DE DATOS RELACIONALES

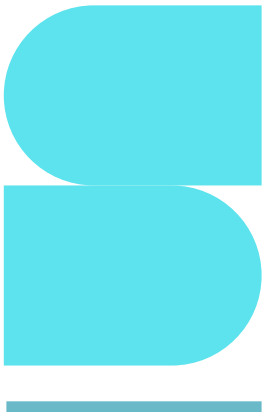
A medida que las aplicaciones modernas requieren mayor escalabilidad, flexibilidad y velocidad en el tratamiento de grandes volúmenes de datos, surge la necesidad de comparar las bases de datos NoSQL con las tradicionales bases de datos relacionales. Este apartado analiza sus principales diferencias en cuanto a estructura, consistencia, escalabilidad, rendimiento y casos de uso, para comprender cuándo es más adecuado emplear uno u otro modelo en función del contexto y los requisitos del sistema.

Característica	Relacional (SQL)	NoSQL
Modelo de datos	Tablas (relaciones)	Clave-valor, documentos, columnas, grafos
Esquema	Fijo, definido con antelación	Flexible o inexistente
Escalabilidad	Vertical (mejorar el hardware)	Horizontal (añadir más nodos)
Lenguaje de consulta	SQL	Depende del motor (propietario o APIs específicas)
Consistencia	Modelo ACID (consistencia fuerte)	Modelo BASE (consistencia eventual)
Ideal para	Aplicaciones tradicionales, datos estructurados	Big Data, apps web escalables, datos no estructurados

4.5. EJEMPLOS DE BASES DE DATOS NOSQL

Nombre	Tipo	Descripción
MongoDB	Documental	Amplio uso, flexible, consultas sobre documentos JSON.
Redis	Clave-valor	Muy rápido, ideal como caché y almacenamiento temporal.
Cassandra	Column-family	Altamente escalable y tolerante a fallos. Usado por grandes empresas.
Neo4j	Basado en grafos	Excelente para modelar relaciones complejas como redes sociales.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

Queda rigurosamente prohibida cualquier forma de reproducción, distribución comunicación pública o transformación total o parcial de esta obra sin el permiso escrito de los titulares de los derechos de explotación.